SeedingMinds

Assessment Name

Java Coding_S3_170126

Module: Coding

Marks: +10 -0

Question 1:

Q46 - Perfect Square or Not

Write a Program to check the Given Number is Perfect Square or Not a perfect Square?

Constraints:

Input :- First Line of Input Consists of One Integer Value.

Output :- Print Given Number is Perfect Square or Not.

Constraints :- Given Input is Zero or Negative Value then Print **InvaliD Input.**

Example:

Input 1 : 10

Output 1: Given Number is Not a Perfect Square.

Input 2 : 5

Output 2: Given Number is Not a Perfect Square.

Input 3 : 16

Output 3: Given Number is a Perfect Square.

Explanation:

Input 1 : 10

Output 1: Given Number is Not a Perfect Square.

Explanation :

Resultant of the **Product of some integer with itself** is nothing but **Perfect Square**.

Given Number is 10 so it is in between the **3*3=9** and **4*4=16**, So it is **Not a Perfect Square.**

Input 2 : 5

Output 2: Given Number is Not a Perfect Square.

Explanation :

Resultant of the **Product of some integer with itself** is nothing but **Perfect Square**.

Given Number is 5 so it is in between the **2*2=4** and **3*3=9**, So it is **Not a Perfect Square**.

Input 3 : 16

Output 3: Given Number is a Perfect Square.

Explanation :

Resultant of the **Product of some integer with itself** is nothing but **Perfect Square.**

Given Number is 16 so it is in between the **4*4=16**, So it is **Perfect Square.**

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	10	Given Number is Not a Perfect Square.
2	5	Given Number is Not a Perfect Square.
3	16	Given Number is a Perfect Square.

Question 2:

Marks: +10 -0

Q2 - Hollow Patterns

Write a Program to Print the following Basic Pattern?

If Input is 4 then Print

```
01 02 03 04

05       06

07       08

09 10 11 12
```

Constraints:

Input :- First Line of Input Consists of One Integer Value (Rows & Columns).

Output :- Print the Square Pattern.

Constraints :- Given Input Must be Greater than Zero or Else Print **"Invalid Input"**.

Example:

Input 1 : 5

Output 1:

```
01 02 03 04

05       06

07       08

09 10 11 12
```

Input 2 : -3

Output 2: Invalid Input

Explanation:

NA

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	4	01 02 03 04 05 06 07 08 09 10 11 12
2	-3	Invalid Input

Question 3:

Marks: +10 -0

Inventory Management System for Stock Analysis

In a bustling marketplace, a store manager needs help keeping track of inventory with precision. The manager frequently receives updates on product quantities, but they come in sporadic lists from various departments. Each list provides product names, followed by their quantities, but these entries are repetitive and sometimes overlap, as different departments update counts independently. Your task is to create a program that reads these inventory updates, consolidates the quantities for each unique product, and provides a clean, summarized list of products along with their total counts. This will help the manager gain accurate insight into stock levels across the store, ensuring smooth operations and quick adjustments to inventory.

Constraints:

- Each store's list can contain up to 1,000 items.
- There can be up to 10 store lists.
- Item names are case-sensitive strings and will contain only letters

Example:

Example 1:

Input: apple:10 banana:20 apple:5 mango:15 banana:5
Output: apple: 15
banana: 25
mango: 15

Example 2:

Input: mango:300 mango:200 avocado:1000 avocado:500 mango:50
Output: avocado: 1500
mango: 550

Explanation:

You are tasked with managing an inventory of products where each product entry consists of an item name and a quantity. Given a list of products input by the user in the format "item:quantity", where each entry is space-separated (e.g., "apple:10 banana:20"), the goal is to parse this list, combine quantities for duplicate items, and then sort the final inventory alphabetically by item name.

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	1apricot:25 banana:50 1apricot:25	1apricot: 50 banana: 50
2	a1pple:50 b@nana:150 a1pple:30 b@nana:50 c#erry:100	a1pple: 80 b@nana: 200 c#erry: 100

Question 4:

Marks: +10 -0

Finding the Closest Element

You are building a recommendation system where the program must search for the closest element in a sorted list of numbers to a given target. The binary search algorithm will be used to efficiently locate the element closest to the target. If two numbers are equidistant to the target, the smaller number is chosen.

Your task is to write a program that uses binary search to find and output the closest number to the target from a sorted list of integers.

Constraints:

- 2 ≤ length of the list ≤ 1000.
- List elements are integers in the range [-10,000, 10,000].
- The list is always sorted in ascending order.

Example:

Example 1:

Input: -10 -5 0 3 9 15

4

Output: 3

2

Example 2:

Input: 1 5 10 15 20

13

Output: 15

Explanation:

You are tasked with writing a program that uses the binary search algorithm to find the number in a sorted list that is closest to a given target value. If two numbers are equidistant to the target, the smaller number is selected. The program will take a sorted list of integers and a target value as input and output the closest number.

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	-20 -15 -10 -5 0 5 -12	-10
2	-10 -8 -6 -4 -2 -7	-8